

Découvrez ce que cela change pour vous



← Découvrez les librairies Python pour la Data Science

 38% de progression

Générez des graphiques complexes avec Seaborn

02:11

Matplotlib offre de nombreuses options pour personnaliser un graphique de A à Z ! Mais il est parfois laborieux de modifier des éléments spécifiques pour obtenir une belle représentation. De plus, une petite modification peut très vite devenir plusieurs lignes de code si on n'y prend pas garde.

Identifiez quand utiliser Seaborn

La librairie **Seaborn** vient proposer une alternative à Matplotlib. C'est également une librairie permettant de générer des graphiques, tout comme Matplotlib. J'irai même plus

loin en exprimant le fait que Seaborn est ce qu'on appelle une **surcouche** à Matplotlib.

Une surcouche correspond à... une couche située au-dessus d'une autre. C'est un terme qui est très régulièrement utilisé dans le monde de la téléphonie mobile, avec les surcouches Android. En gros, Android est la première couche disponible sur les Smartphones, et la surcouche du constructeur se rajoute par-dessus, offrant un nouveau design et de nouvelles fonctionnalités.

Seaborn est exactement pareil vis-à-vis de Matplotlib. La librairie reprend les mêmes principes que Matplotlib, mais en ajoutant quelque chose "par-dessus", afin d'offrir des qualités de graphiques différents, des nouvelles fonctionnalités, etc. Si vous me permettez une dernière image, on pourrait imaginer que Seaborn est une voiture flambant neuve dont l'ensemble du châssis et le moteur seraient Matplotlib.

Et du coup, quelles sont ces nouvelles fonctionnalités offertes par Seaborn ?

Seaborn vient corriger plusieurs défaut de Matplotlib :

- il propose de multiples modèles graphiques prédéfinis de bonne qualité esthétique, en modifiant les options graphiques par défaut de Matplotlib ;
- il ajoute une interaction avec les data frames afin de faciliter grandement la génération de graphiques à partir de ceux-ci ;
- il propose un catalogue – très – dense de fonctions graphiques pour répondre le plus précisément possible à une problématique donnée.

En contrepartie, Seaborn propose un peu moins d'options de personnalisation en tant que telles.

MAIS, comme Seaborn est juste une surcouche de Matplotlib, il est possible d'utiliser l'ensemble des options de personnalisation vues dans le précédent chapitre, directement sur des graphiques tracés avec Seaborn. Extraordinaire, non ?

Les deux librairies se complètent pour exploiter au mieux les avantages de l'une et de l'autre. C'est un mariage particulièrement harmonieux, que je vous invite à découvrir.

Prenez en main Seaborn

Seaborn étant une surcouche de Matplotlib, il y a donc de nombreuses ressemblances entre les deux librairies. Vous souhaitez réaliser un nuage de points ? Nous avons la fonction `scatter` avec Matplotlib, laissez-moi vous présenter la fonction...

`scatterplot` avec Seaborn.

Voilà comment l'utiliser :

```
1 import seaborn as sns
2 sns.scatterplot(data=prets, x='revenu', y='taux_endettement')
```

Explicitons un peu son fonctionnement :

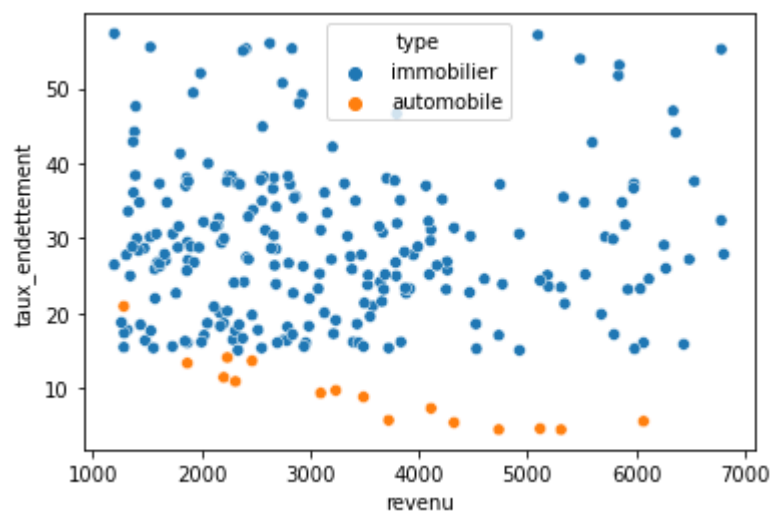
- dans un premier temps, on définit le data frame qui va être utilisé pour tracer le graphique, via l'argument `data` ;
- ensuite, il suffit de fixer les variables du data frame défini, à mettre en abscisse et en ordonnée.

Et le réel intérêt de Seaborn apparaît lorsqu'on essaie d'ajouter des dimensions supplémentaires, comme des couleurs ou des tailles de points.

Reprenons le même graphique en ajoutant en couleur le type de graphique tracé. Là où on avait besoin d'une boucle avec Matplotlib, voilà le code avec Seaborn :

```
1 sns.scatterplot(data=prets, x='revenu', y='taux_endettement', hue='type')
```

... pour obtenir :



Application de la fonction scatterplot avec Seaborn

En précisant la variable via l'argument `hue` , Seaborn crée automatiquement une couleur pour chaque valeur existante.

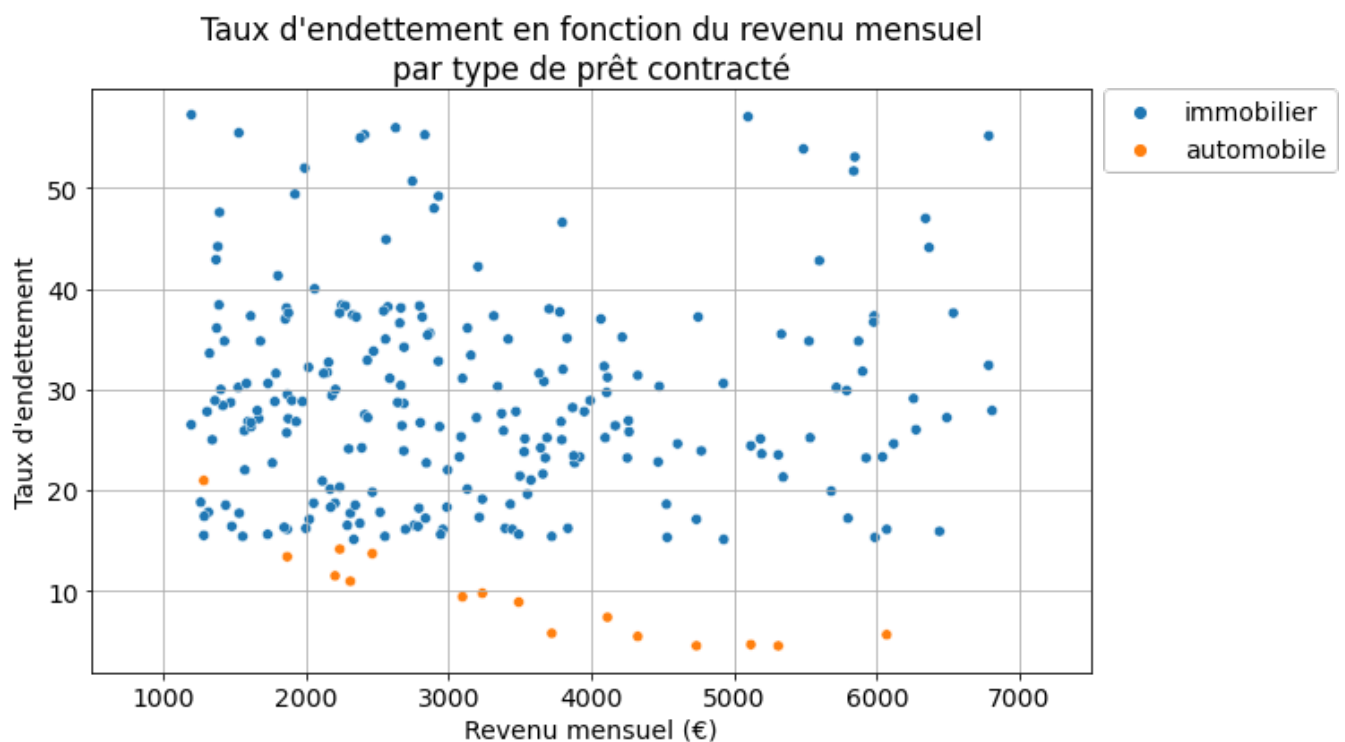
À présent, plutôt que l'information du type de prêt, on souhaite ajouter l'information de la durée du prêt. On peut faire cela en associant la taille d'un point à la durée d'un prêt :

```
1 sns.scatterplot(data=prets, x='revenu', y='taux_endettement', size='duree')
```

Le nombre d'informations à afficher n'est pas limité, on pourrait très bien mixer la couleur avec la taille des points ! Cependant, attention à ne pas sacrifier la clarté et la lisibilité d'un graphique en voulant y ajouter "trop" d'informations.

Pour finaliser ce graphique, on peut utiliser les différentes fonctions de Matplotlib vues précédemment :

```
1 plt.figure(figsize=(10,6))
2
3 plt.rcParams.update({'font.size': 14})
4
5 sns.scatterplot(data=prets, x='revenu', y='taux_endettement', hue='type')
6
7 plt.ylabel("Taux d'endettement")
8
9 plt.xlabel('Revenu mensuel (€)')
10
11 plt.grid()
12
13 plt.xlim(500, 7500)
14
15 plt.legend(bbox_to_anchor=(1, 1.02))
16
17 plt.title("Taux d'endettement en fonction du revenu mensuel\npar type de prêt contracté")
18
19 plt.show()
```



Application d'options à la fonction scatterplot avec Seaborn

Explicitons les différentes fonctions utilisées :

1. La fonction `rcParams.update` , avec l'argument `font.size` , définit pour tous les graphiques – et non pas uniquement le graphique actuel – la taille de la police de tous les éléments (xlabel, title, legend, etc.) à 14.
2. `grid` permet d'afficher le quadrillage.
3. Via la fonction `xlim` , on fixe les limites des graduations de 500 à 7 500.
4. La fonction `legend` affiche la légende sur le graphique. La petite nouveauté ici est

l'utilisation de l'argument `bbox_to_anchor` permettant de fixer la position de la légende en dehors du graphique, aux coordonnées renseignées.

Agrégez des données avec Seaborn

Reprenons à présent le diagramme à barres du chiffre d'affaires par agence, réalisé précédemment. Avec Matplotlib, avant de tracer ce dernier, on avait eu besoin au préalable d'agréger nos données. La fonction `barplot` de Seaborn nous permet de réaliser cette étape directement à l'intérieur du graphique :

```
1 sns.barplot(data=prets, x='ville', y='remboursement', errorbar='ci', estimator=sum)
```

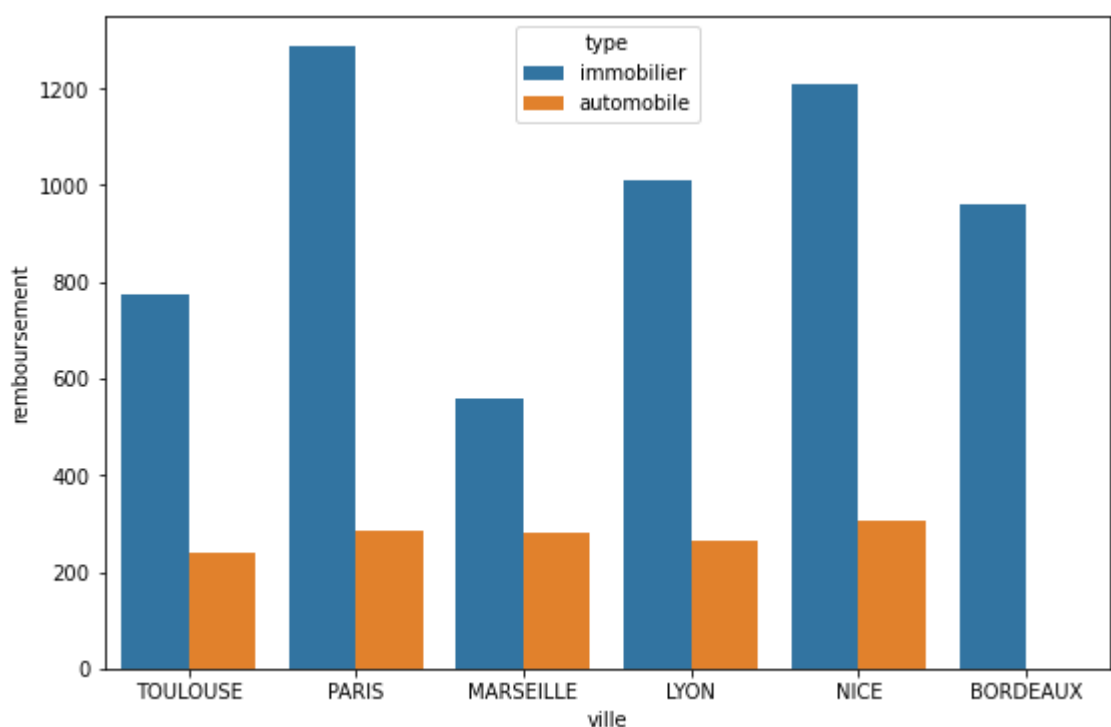
Le début est similaire à ce que nous avons vu avec la fonction `scatterplot`, mais nous découvrons ici deux nouveaux arguments :

- `errorbar`, pour ajouter les intervalles de confiance ("ci") à chaque barre. Fixer cet argument à `None` permet de les faire disparaître ;
- `estimator` correspond à la fonction d'agrégation.

Tous les arguments vus précédemment fonctionnent également, notamment avoir des sous-groupes – par exemple : par type de prêt – en couleur :

```
1 sns.barplot(data=prets, x='ville', y='remboursement', errorbar='ci', estimator=np.mean,  
hue='type')
```

... pour le résultat suivant :



Application de la fonction barplot avec Seaborn

Seaborn possède de nombreuses fonctions (plus d'une vingtaine) pour répondre à toutes les situations possibles en termes de visualisation. Vous pouvez trouver la liste exhaustive sur [la documentation officielle](#). Mais pour en citer quelques-unes rapidement :

- `lineplot` pour tracer des courbes ;
- `boxplot` pour des boîtes à moustaches ;
- `histplot` pour des histogrammes ;
- `kdeplot` pour un graphique à densité ;
- `pairplot` pour tracer l'ensemble des variables numériques deux à deux, sur un seul graphique !

N'hésitez pas à tester toutes ces fonctions de votre côté pour apprécier la plus-value apportée par Seaborn.

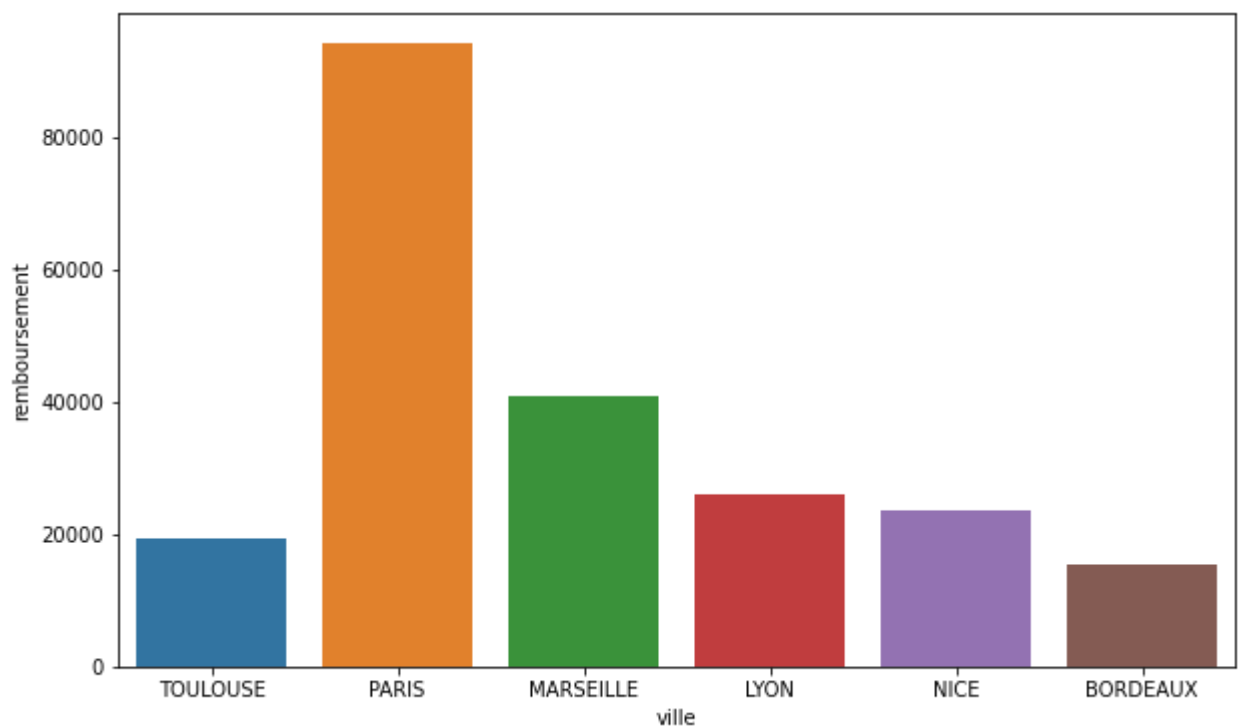
Améliorez l'aspect graphique de vos visualisations

Enfin, je ne peux pas terminer ce cours sans vous parler de la façon dont Seaborn gère l'aspect graphique de ses visualisations. Je vous ai expliqué un peu plus tôt que Seaborn apportait un vrai plus esthétique par rapport à Matplotlib. Deux aspects permettent cela : les **palettes graphiques** et les **thèmes**.

Les palettes graphiques

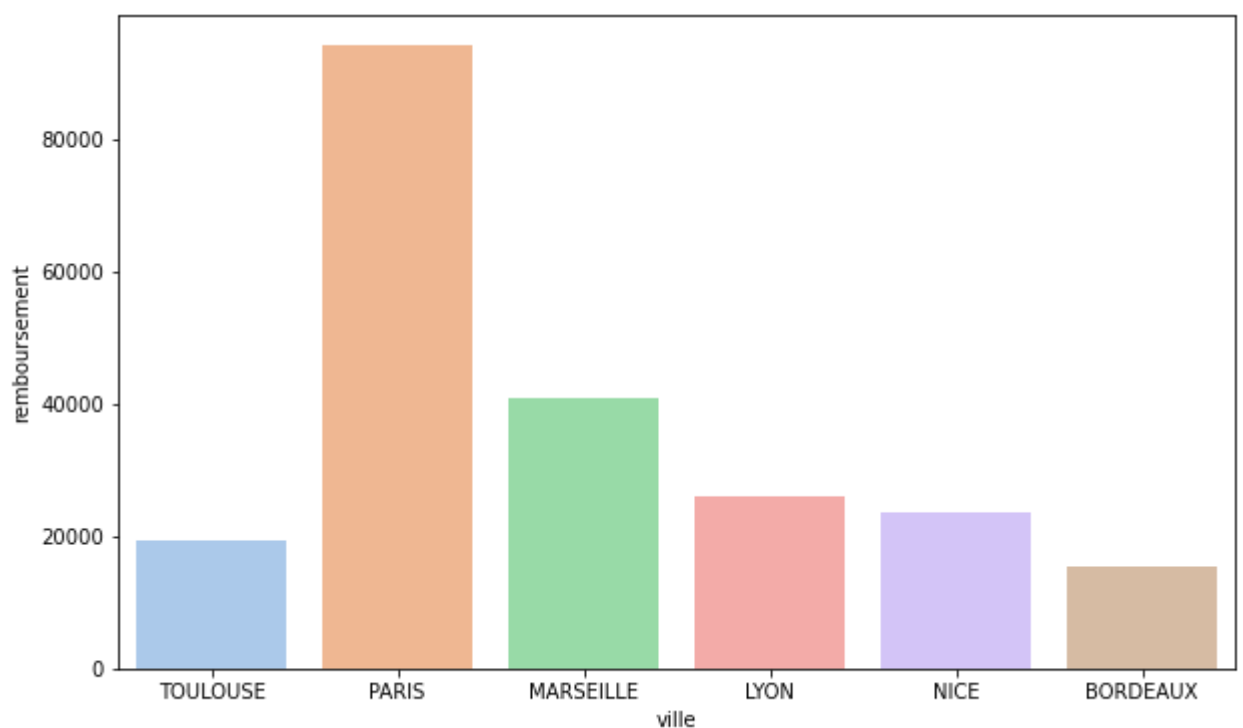
Les **palettes graphiques** correspondent à un ensemble de couleurs qui vont être utilisées dans un graphique. Vous pouvez naturellement les définir vous-même, mais Seaborn propose plusieurs palettes, utilisables à votre guise, qui sont particulièrement harmonieuses.

Partons du graphique présentant le chiffre d'affaires total par agence :



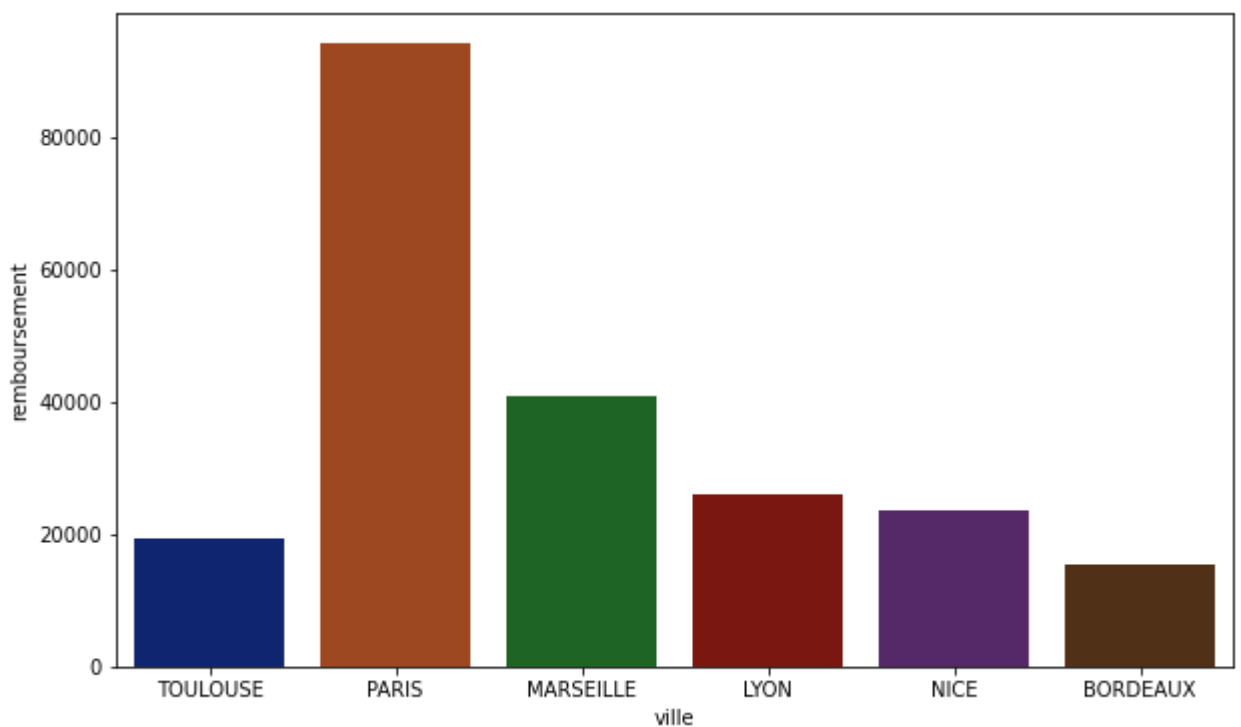
Graphique à améliorer avec Seaborn

Pour fixer une nouvelle palette, on utilise la fonction `set_palette` de Seaborn. Donnons à notre graphiques un aspect un peu plus *pastel* : `sns.set_palette('pastel')` :



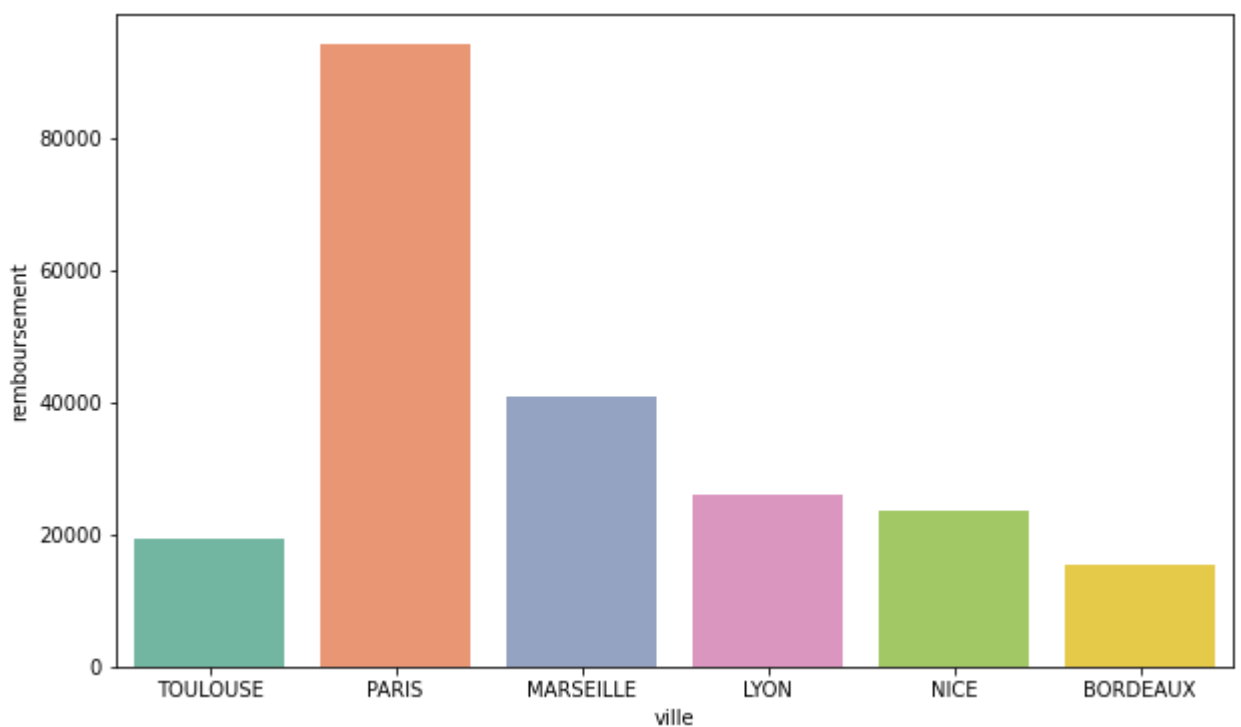
Application de pastel avec la fonction `set_palette`

Ou un aspect un peu plus sombre, via `sns.set_palette('dark')` :



Application de dark avec la fonction set_palette

Il y a aussi plusieurs ensembles (ou sets, en anglais) à disposition. J'aime particulièrement le second `sns.set_palette('Set2')` :



Application d'un ensemble avec la fonction set_palette

Vous pourrez retrouver toutes les informations concernant les différentes palettes possibles, directement sur [la page de couleurs correspondante](#).

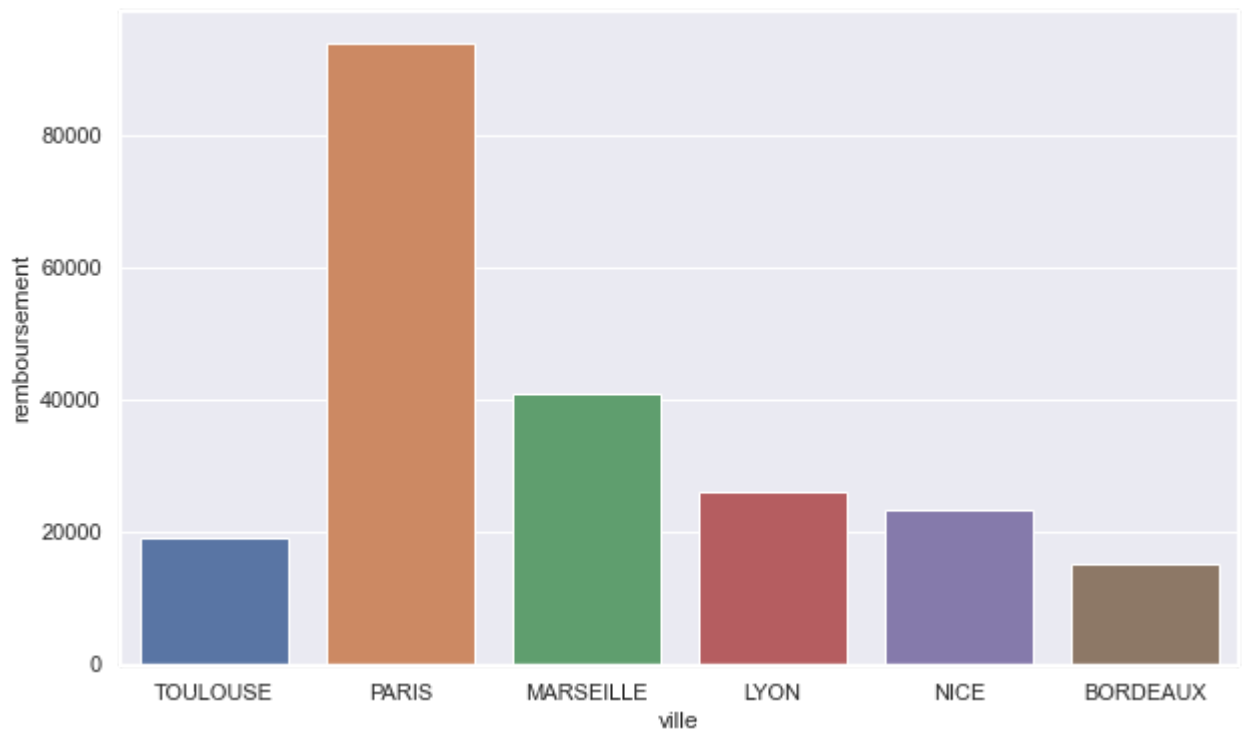
Les thèmes

Les **thèmes** correspondent à des ensembles d'aspects graphiques (sur les axes, le fond, la graduation, etc.) prédéfinis par Seaborn et utilisables à notre guise. On peut choisir un

thème à appliquer via la fonction `set_theme` .

Appliquons dans un premier temps le set de base :

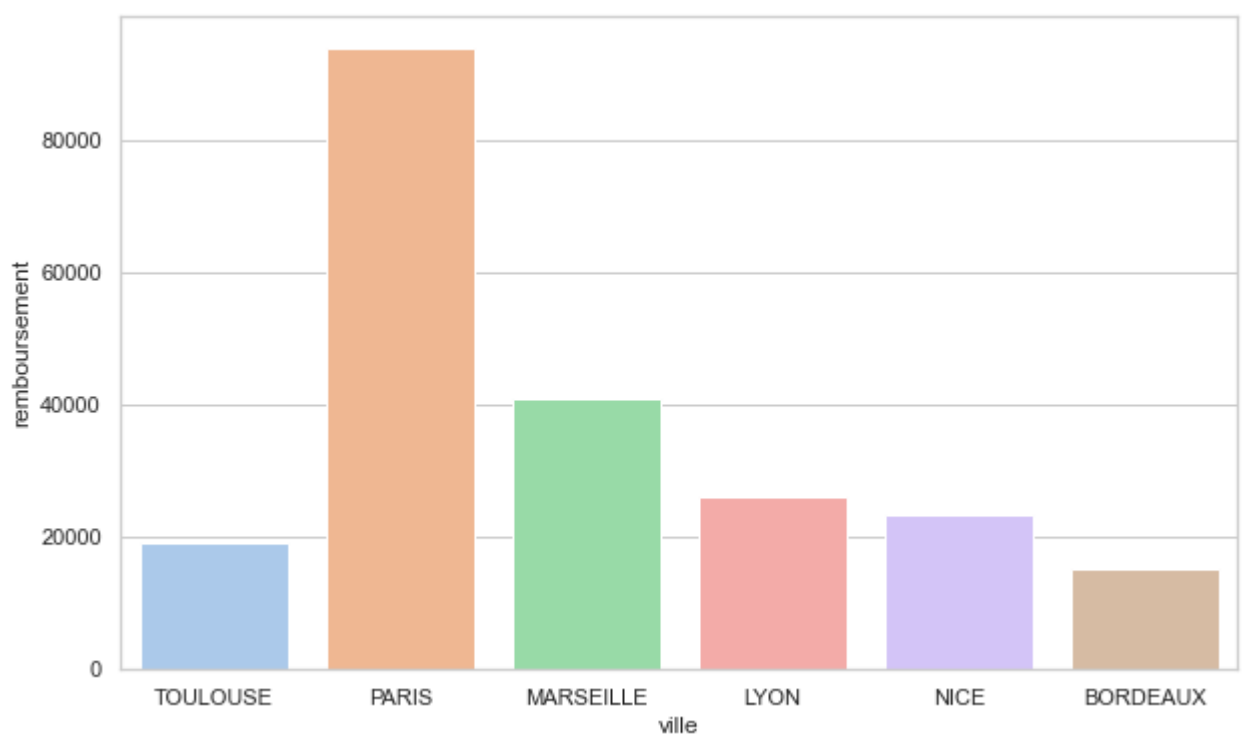
```
1 sns.set_theme()
```



Application du set de base avec la fonction `set_theme`

L'aspect est tout de suite complètement différent, n'est-ce pas ? On peut même mixer le choix du thème avec la palette de couleurs :

```
1 sns.set_theme(style='whitegrid', palette='pastel')
```



Mix entre thème et palette de couleurs

En plus de parfaitement intégrer les data frames dans le processus de génération de graphiques, Seaborn permet de créer de jolis graphiques de façon très simple. Et si un aspect ne nous plaît pas sur un thème en particulier ? Aucun problème ! Nous pouvons appliquer les différentes fonctions de Matplotlib pour pousser encore plus loin la personnalisation. Ces deux librairies forment une boîte à outils complète en termes de datavisualisation.

Que diriez-vous à présent d'appliquer vos nouvelles connaissances acquises sur Seaborn ? C'est ce que je vous propose dans l'exercice suivant.

À vous de jouer



Contexte

Habituellement, les graphiques du reporting mensuel sont réalisés avec Matplotlib. Votre ultime tâche va être d'améliorer le visuel de ces derniers avec Seaborn.

Consignes

L'objectif va être de reprendre le travail qui avait été réalisé précédemment, avec Seaborn. En plus de ces graphiques, quelques demandes additionnelles ont été glissées par votre manager, notamment pour ajouter des dimensions aux différents graphiques.

Je vous invite à vous rendre sur [l'exercice](#).

Vérifiez votre travail

Top !! Que diriez vous de comparer à présent le travail réalisé avec [la correction](#) ?

En résumé

- Seaborn est une surcouche de Matplotlib ; il est donc possible de mixer des fonctions des deux librairies sur un seul et même graphique.
- Seaborn est né pour faciliter la création de visualisations à partir des data frames Pandas. Grâce à cette philosophie, on peut ajouter autant de dimensions (couleur, forme, taille, etc.) que de variables dans un data frame.
- Seaborn ajoute une dimension esthétique à ses graphiques, en donnant la possibilité de changer les thèmes et/ou les palettes de couleurs.

Bravo ! Vous êtes parvenu au bout de ce cours. Mais avant de nous quitter, je vous propose de tester vos connaissances dans le quiz clôturant cette partie.

 [Avez-vous une suggestion pour nous ?](#) 

Faire le quiz →

Construire des data visualisations avec Matplotlib et Seaborn